# xv6 Bonus Report

Deploying `sequential_compute` and `parallel_compute` on xv6

**Bonus Task**

Deploy the same project with the same specifications in xv6 after making the necessary changes. Create 2 user programs that use them.

xv6-public (x86)

# Contents

**1 Key Differences Between Linux and xv6** **2**

**2 Files Created** **2**

**3 Source Code** **2**
3.1 `parse.h` — File Parser for xv6 . . . 3
3.2 `compute.h` — Type and Function Declarations . . . 4
3.3 `compute.c` — Core Compute Logic . . . 4
3.4 `seq_app.c` — User Program 1 . . . 6
3.5 `par_app.c` — User Program 2 . . . 7

**4 Makefile Changes** **7**
4.1 Adding programs to UPROGS . . . 7
4.2 Embedding the data file and explicit link rules . . . 7

**5 Build and Execution** **8**
5.1 Building . . . 8
5.2 Running inside xv6 . . . 8

# 1 Key Differences Between Linux and xv6

Before writing any code, we must understand what xv6 lacks compared to a standard Linux environment. The table below summarises every adaptation made.

| Feature | Linux | xv6 (what we do instead) |
|---|---|---|
| `wait() signature` | `wait(NULL)` | `wait()` — takes no arguments |
| `printf() signature` | `printf("...", ...)` | `printf(1, "...", ...)` — first arg is file descriptor |
| `exit() signature` | `exit(0)` | `exit()` — no status code |
| `Standard headers` | `<stdio.h>`, `<stdlib.h>`, etc. | `types.h` + `user.h` |
| `strtok, isdigit` | Available in `<string.h>`, `<ctype.h>` | Not available — we write our own parser |
| `File I/O` | `fopen/fgets/fclose` | `open/read/close` syscalls |

Table 1: Linux vs. xv6 differences and our solutions

# 2 Files Created

Five new files were added to `~/xv6-public/`:

| File | Purpose |
|---|---|
| `parse.h` | Reads integers from a file using xv6 syscalls; replaces the Linux version that used `strtok` and `fgets` |
| `compute.h` | Declares `func_t`, `sequential_compute`, and `parallel_compute` (unchanged from Linux) |
| `compute.c` | Implements both compute functions, adapted for xv6 |
| `seq_app.c` | User program 1 — calls `sequential_compute` |
| `par_app.c` | User program 2 — calls `parallel_compute` |

Table 2: New files added to xv6-public

One data file was also added:

- `numbers.txt` — contains the integers `1 2 3 4 5`, one per line, used to test both programs.

# 3 Source Code

### 3.1 parse.h — File Parser for xv6

The Linux version of `parse.h` relied on `fgets`, `strtok`, and `isdigit` from the standard C library. None of these exist in xv6. The replacement uses the raw `open`/`read`/`close` syscalls to load the file into a heap buffer, then walks the buffer byte-by-byte to extract integers. The helper `is_digit()` replaces `isdigit()`. Dynamic resizing with `malloc`/`realloc` still works because xv6 provides these in `umalloc.c`.

```c
#ifndef PARSE_H
#define PARSE_H

#include "types.h"
#include "user.h"

static int is_digit(char c) { return c >= '0' && c <= '9'; }

static int *read_numbers(const char *filepath, int *count) {
    int fd = open(filepath, 0);   /* O_RDONLY = 0 in xv6 */
    if (fd < 0) {
        printf(1, "Cannot open %s\n", filepath);
        return 0;
    }

    /* Read the whole file into a heap buffer */
    int cap = 1024;
    char *filebuf = malloc(cap);
    int total = 0, n;
    char tmp[512];

    while ((n = read(fd, tmp, 512)) > 0) {
        if (total + n >= cap) {
            cap *= 2;
            char *nb = malloc(cap);
            memmove(nb, filebuf, total);
            free(filebuf);
            filebuf = nb;
        }
        memmove(filebuf + total, tmp, n);
        total += n;
    }
    close(fd);

    /* Parse integers from the buffer */
    int num_cap = 64;
    int *nums = malloc(num_cap * sizeof(int));
    *count = 0;

    int i = 0;
    while (i < total) {
        char c = filebuf[i];
        if (is_digit(c) ||
            (c == '-' && i+1 < total && is_digit(filebuf[i+1]))) {
            int neg = 0;
            if (c == '-') { neg = 1; i++; }
            int val = 0;
            while (i < total && is_digit(filebuf[i]))
                val = val * 10 + (filebuf[i++] - '0');
```

```
            if (*count == num_cap) {
                num_cap *= 2;
                int *tmp2 = malloc(num_cap * sizeof(int));
                memmove(tmp2, nums, *count * sizeof(int));
                free(nums);
                nums = tmp2;
            }
            nums[(*count)++] = neg ? -val : val;
        } else {
            i++;
        }
    }

    free(filebuf);
    return nums;
}

#endif
```

Listing 1: `parse.h` — xv6 integer file parser

### 3.2 `compute.h` — Type and Function Declarations

This header is essentially unchanged from the Linux version. The only adaptation is using xv6 headers instead of standard C headers.

```
#ifndef COMPUTE_H
#define COMPUTE_H

typedef int (*func_t)(int, int);

int sequential_compute(const char *filepath, func_t f);
int parallel_compute(const char *filepath, int n_proc, func_t f);

#endif
```

Listing 2: `compute.h` — shared declarations

### 3.3 `compute.c` — Core Compute Logic

The logic of both functions is **identical** to the Linux version. The three adaptations are:

1. `wait()` takes no arguments in xv6 (declared as `int wait(void)` in `user.h`).
2. `printf(1, ...)` requires a file-descriptor as first argument.
3. `exit()` takes no status code.

```
#include "types.h"
#include "user.h"
#include "compute.h"
#include "parse.h"

int sequential_compute(const char *filepath, func_t f) {
    int count = 0;
    int *nums = read_numbers(filepath, &count);
```

```c
    if (!nums || count == 0) { free(nums); return 0; }

    int result = nums[0];
    for (int i = 1; i < count; i++)
        result = f(result, nums[i]);

    free(nums);
    return result;
}

int parallel_compute(const char *filepath, int n_proc, func_t f) {
    int count = 0;
    int *nums = read_numbers(filepath, &count);
    if (!nums || count == 0) { free(nums); return 0; }

    if (n_proc > count) n_proc = count;

    /* Flat pipe arrays: ctrl_pipe[i*2] and ctrl_pipe[i*2+1] */
    int *ctrl_pipe = malloc(n_proc * 2 * sizeof(int));
    int *res_pipe  = malloc(n_proc * 2 * sizeof(int));

    for (int i = 0; i < n_proc; i++) {
        pipe(&ctrl_pipe[i*2]);
        pipe(&res_pipe[i*2]);
    }

    int chunk     = count / n_proc;
    int remainder = count % n_proc;

    for (int i = 0; i < n_proc; i++) {
        int pid = fork();
        if (pid < 0) { printf(1, "fork failed\n"); exit(); }

        if (pid == 0) {
            /* ===== CHILD ===== */
            for (int j = 0; j < n_proc; j++) {
                close(ctrl_pipe[j*2+1]);
                if (j != i) close(ctrl_pipe[j*2+0]);
                close(res_pipe[j*2+0]);
                if (j != i) close(res_pipe[j*2+1]);
            }

            int start, end;
            read(ctrl_pipe[i*2+0], &start, sizeof(int));
            read(ctrl_pipe[i*2+0], &end,   sizeof(int));
            close(ctrl_pipe[i*2+0]);

            int partial = nums[start];
            for (int k = start + 1; k <= end; k++)
                partial = f(partial, nums[k]);

            write(res_pipe[i*2+1], &partial, sizeof(int));
            close(res_pipe[i*2+1]);

            free(nums); free(ctrl_pipe); free(res_pipe);
            exit();
            /* ===== END CHILD ===== */
        }
```

```c
    }

    /* ===== PARENT ===== */
    for (int i = 0; i < n_proc; i++) {
        int start = i * chunk;
        int end   = start + chunk - 1;
        if (i == n_proc - 1) end += remainder;

        write(ctrl_pipe[i*2+1], &start, sizeof(int));
        write(ctrl_pipe[i*2+1], &end,   sizeof(int));
        close(ctrl_pipe[i*2+1]);
        close(ctrl_pipe[i*2+0]);
        close(res_pipe[i*2+1]);
    }

    /* wait() in xv6 takes NO arguments */
    for (int i = 0; i < n_proc; i++)
        wait();

    int *partials = malloc(n_proc * sizeof(int));
    for (int i = 0; i < n_proc; i++) {
        read(res_pipe[i*2+0], &partials[i], sizeof(int));
        close(res_pipe[i*2+0]);
    }

    int result = partials[0];
    for (int i = 1; i < n_proc; i++)
        result = f(result, partials[i]);

    free(partials); free(nums); free(ctrl_pipe); free(res_pipe);
    return result;
}
```

Listing 3: `compute.c` — sequential and parallel compute for xv6

### 3.4 `seq_app.c` — User Program 1

This program takes a filename and an operation (`add` or `mul`) as command-line arguments, calls `sequential_compute`, and prints the result. It demonstrates the sequential version running entirely inside a single xv6 process.

```c
#include "types.h"
#include "user.h"
#include "compute.h"

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf(1, "Usage: seq_app <file> <add|mul>\n");
        exit();
    }
    func_t f = (argv[2][0] == 'a') ? add : mul;
    int result = sequential_compute(argv[1], f);
    printf(1, "sequential_compute result: %d\n", result);
    exit();
```

```
}
```

Listing 4: `seq_app.c` — user program 1: sequential compute

### 3.5 `par_app.c` — User Program 2

This program takes a filename, a process count, and an operation as command-line arguments, calls `parallel_compute`, and prints the result.

```c
#include "types.h"
#include "user.h"
#include "compute.h"

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main(int argc, char *argv[]) {
    if (argc < 4) {
        printf(1, "Usage: par_app <file> <n_proc> <add|mul>\n");
        exit();
    }
    int n_proc = atoi(argv[2]);
    func_t f   = (argv[3][0] == 'a') ? add : mul;
    int result = parallel_compute(argv[1], n_proc, f);
    printf(1, "parallel_compute result (n_proc=%d): %d\n", n_proc,
   result);
    exit();
}
```

Listing 5: `par_app.c` — user program 2: parallel compute

## 4 Makefile Changes

Two changes were made to `Makefile`.

### 4.1 Adding programs to UPROGS

The `UPROGS` variable tells xv6's build system which user programs to compile and embed into the filesystem image. The two new entries were appended:

```
UPROGS=\
    ...
    _test\
    _seq_app\
    _par_app\
```

Listing 6: UPROGS additions in Makefile

### 4.2 Embedding the data file and explicit link rules

The `fs.img` rule was updated to include `numbers.txt` so it is available inside the running xv6 filesystem:

```
fs.img: mkfs README $(UPROGS)
    ./mkfs fs.img README numbers.txt $(UPROGS)
```

Listing 7: fs.img rule updated to include numbers.txt

# 5 Build and Execution

## 5.1 Building

```
cd ~/xv6-public
make clean
make qemu
```

Listing 8: Build commands

## 5.2 Running inside xv6

Both programs are run against `numbers.txt` which contains `1 2 3 4 5`:

```
$ seq_app numbers.txt add
sequential_compute result: 15

$ par_app numbers.txt 2 add
parallel_compute result (n_proc=2): 15
```

Listing 9: xv6 shell session

Both results are correct: $1 + 2 + 3 + 4 + 5 = 15$, confirming that the logic is preserved across the port.

With `n_proc=2` and 5 numbers: child 0 receives indices $[0, 1]$ and computes $1 + 2 = 3$; child 1 receives indices $[2, 4]$ (last child gets the remainder) and computes $3 + 4 + 5 = 12$; the parent aggregates $3 + 12 = 15$.